ANÁLISIS DEL RETO

Juan Sebastian Rojas Marin, js.rojasm123@uniandes.edu.co, 202610140

Katy Teran Mercado, k.teranm@uniandes.edu.co, 202614247

Juan Jose Rincon Santos, jj.rincons1@uniandes.edu.co, 202615789

Requerimiento 1

Descripción

```python
def sort_crit_req_1(sale1, sale2):
    """
    Ascendente por descuento; empate -> fecha más antigua primero;
    empate -> menor order_id.
    """
    if sale1["discount_percent"] != sale2["discount_percent"]:
        return sale1["discount_percent"] < sale2["discount_percent"]
    if sale1["order_date"] != sale2["order_date"]:
        return sale1["order_date"] < sale2["order_date"]
    return order_number(sale1["order_id"]) < order_number(sale2["order_id"])
```

```python
def req_1(catalog, brand, discount_min, discount_max):
    """
    Retorna el resultado del requerimiento 1: órdenes de una marca con
    descuento dentro de [discount_min, discount_max].
    """
    start = get_time()
    result = al.new_list()
    total_cost = 0

    tree = mlp.get(catalog["brand_discount"], brand.strip().lower())
    if tree is not None and discount_min <= discount_max:
        # Rango en el RBT: listas de ventas con descuento en [min, max]
        buckets = rbt.values(tree, float(discount_min), float(discount_max))
        for i in range(al.size(buckets)):
            bucket = al.get_element(buckets, i)
            for j in range(al.size(bucket)):
                sale = al.get_element(bucket, j)
                al.add_last(result, sale)
                total_cost += sale["final_price_usd"]
        al.merge_sort(result, sort_crit_req_1)

    end = get_time()

    return {
        "time": delta_time(start, end),
        "total": al.size(result),
        "total_cost": total_cost,
        "orders": result,
    }
```

Consulta las órdenes de una marca cuyo porcentaje de descuento está dentro de un rango inclusivo, sin recorrer la lista principal. Durante la carga se construye la tabla de hash brand_discount (Linear Probing). Su llave es la marca en minúsculas y su valor es un árbol rojo-negro (RBT) con llave discount_percent y valor un array_list con las ventas que tienen ese descuento. Con la marca normalizada se obtiene su árbol en una sola operación, y con rbt.values se sacan solo las listas cuyo descuento cae en [discount_min, discount_max]. Al recorrerlas se agregan las ventas al resultado y en la misma pasada se acumula el costo total (final_price_usd). Después se ordena con Merge Sort: descuento ascendente, luego fecha más antigua y luego menor order_id. Si la marca no existe o el rango no es válido, se retorna una lista vacía sin error.

Entrada	catalog, brand, discount_min, discount_max
Salidas	time, total, total_cost, orders
Implementado (Sí/No)	Implementado por Juan Rojas.

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	**Complejidad**
Normalizar la marca y buscarla en brand_discount (mlp.get, Linear Probing)	O(1) promedio
Consultar el rango de descuentos en el RBT de la marca (rbt.values)	O(log d + r)
Recorrer las k ventas de las r listas obtenidas y acumular total_cost	O(k)
Ordenar las k ventas con Merge Sort	O(k log k)
TOTAL	***O(log d + k log k)***

d = descuentos distintos de la marca, r = descuentos dentro del rango, k = ventas que cumplen (r ≤ k).

Análisis

Análisis de resultados de la implementación, tener cuenta las pruebas realizadas y el analisis de complejidad.

Indexar por marca y después por descuento evita recorrer todo el archivo. La tabla de hash da acceso O(1) a la marca, y el RBT hace la búsqueda por rango en O(log d + r): baja solo por las ramas que intersecan el rango y descarta el resto. Así el costo depende del tamaño del resultado y no del número total de ventas. Se usó un árbol balanceado porque el filtro es por intervalo, algo que una tabla de hash no resuelve sin revisar todas las llaves. El costo dominante es Merge Sort, O(k log k), que se usa porque es estable y garantiza ese orden en el peor caso.

ANÁLISIS DEL RETO

Juan Sebastian Rojas Marin, js.rojasm123@uniandes.edu.co, 202610140

Katy Teran Mercado, k.teranm@uniandes.edu.co, 202614247

Juan Jose Rincon Santos, jj.rincons1@uniandes.edu.co, 202615789

Requerimiento <<2>>

Descripción

```python
def req_2(catalog, country, rating_min, rating_max):
    """
    Retorna el resultado del requerimiento 2: órdenes de un país con
    calificación del cliente dentro de [rating_min, rating_max].
    """
    start = get_time()

    result = al.new_list()
    total_cost = 0.0

    tree = mlp.get(catalog["country_rating"], country.strip().lower())

    if tree is not None and rating_min <= rating_max:
        # Rango en el RBT: listas de ventas con calificación en [min,max]
        buckets = rbt.values(tree, float(rating_min), float(rating_max))
        for i in range(al.size(buckets)):
            bucket = al.get_element(buckets, i)

            for j in range(al.size(bucket)):
                sale = al.get_element(bucket, j)

                al.add_last(result, sale)
                total_cost += sale["final_price_usd"]

        al.merge_sort(result, sort_crit_req_2)

    end = get_time()

    return {
        "time": delta_time(start, end),
        "total": al.size(result),
        "total_cost": total_cost,
        "orders": result,
    }
```

```python
def sort_crit_req_2(sale1, sale2):
    """
    Descendente por calificación; empate -> fecha más reciente primero.
    """

    if sale1["customer_rating"] != sale2["customer_rating"]:
        return sale1["customer_rating"] > sale2["customer_rating"]

    return sale1["order_date"] > sale2["order_date"]
```

Este requerimiento retorna las órdenes de venta de un país (country) cuya calificación del cliente está entre rating_min y rating_max. En la carga de datos se crea el mapa catalog["country_rating"], donde la llave es el país y el valor es un árbol RBT ordenado por customer_rating; cada nodo del árbol guarda la lista de ventas que tienen esa calificación. Lo primero que se hace es obtener del mapa el árbol del país (tree) y, con rbt.values(tree, rating_min, rating_max), sacar solo las listas (buckets) de las calificaciones que están dentro del rango. Luego se recorren los buckets para agregar cada venta (sale) a result y sumar su final_price_usd en total_cost. Por último, result se ordena con al.merge_sort usando sort_crit_req_2, que ordena de mayor a menor customer_rating y, si hay empate, de la order_date más reciente a la más antigua.

Entrada	catalog, country, rating_min, rating_max
Salidas	time, total, total_cost, orders
Implementado (Sí/No)	Sí. Implementado por Katy Yulieth Teran Mercado

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	**Complejidad**
Obtener el árbol del país	O(1)
Obtener los buckets en el rango	O(log n + k)
Recorrer los buckets, agregar a result	O(k)
Ordenar result	O(k log k)
TOTAL	***O(k log k)***

Análisis

Lo que más influye en el tiempo de este requerimiento es k, el número de órdenes que cumplen el filtro, y no el total de ventas. Esto se debe a que, gracias a la estructura que armé en la carga, no tengo que revisar ventas que no sirven: encontrar el árbol del país es constante y llegar al rango de calificaciones es logarítmico, así que casi todo el trabajo se hace sobre las órdenes que sí van en la respuesta. Recorrerlas cuesta O(k), pero lo que domina es el ordenamiento final con merge sort, que cuesta O(k log k). Por eso la complejidad del requerimiento es O(k log k). El peor caso sería que todas las ventas fueran del país y estuvieran dentro del rango: ahí k sería igual a n y la complejidad llegaría a O(n log n).

Las pruebas confirman este comportamiento. Usé siempre USA, 3.5 y 5.0, y promedié varias ejecuciones con cada archivo: con el small obtuve 967 órdenes en 6.42 ms, con el medium 1 913 en 11.76 ms y con el large 3 839 en 27.59 ms. De small a large, k creció casi 4 veces y el tiempo un poco más de 4. Ese “poco más” es justamente el factor log k que agrega el ordenamiento.

ANÁLISIS DEL RETO

Juan Sebastian Rojas Marin, js.rojasm123@uniandes.edu.co, 202610140

Katy Teran Mercado, k.teranm@uniandes.edu.co, 202614247

Juan Jose Rincon Santos, jj.rincons1@uniandes.edu.co, 202615789

Requerimiento <<3>>

Descripción

```python
def req_3(catalog, channel, category, price_min, price_max):
    """
    Retorna el resultado del requerimiento 3
    """
    start = get_time()
    result = al.new_list()
    total_cost = 0.0

    cc_key = channel.strip().lower() + "|" + category.strip().lower()
    tree = mlp.get(catalog["channel_category_price"], cc_key)
    if tree is not None and price_min <= price_max:
        buckets = rbt.values(tree, float(price_min), float(price_max))
        for i in range(al.size(buckets)):
            bucket = al.get_element(buckets, i)
            for j in range(al.size(bucket)):
                sale = al.get_element(bucket, j)
                al.add_last(result, sale)
                total_cost += sale["final_price_usd"]
        al.merge_sort(result, sort_crit_req_3)

    end = get_time()

    return {
        "time": delta_time(start, end),
        "total": al.size(result),
        "total_cost": total_cost,
        "orders": result,
    }
```

```python
def sort_crit_req_3(sale1, sale2):
    """
    Ascendente por precio final; empate -> fecha mas antigua primero;
    empate -> menor order_id.
     """
    if sale1["final_price_usd"] != sale2["final_price_usd"]:
        return sale1["final_price_usd"] < sale2["final_price_usd"]
    if sale1["order_date"] != sale2["order_date"]:
        return sale1["order_date"] < sale2["order_date"]
    return order_number(sale1["order_id"]) < order_number(sale2["order_id"])
```

Este requerimiento lista las órdenes de venta de un canal de venta (channel) y una categoría de producto (category) cuyo precio final está entre price_min y price_max. En la carga de datos se crea el mapa catalog["channel_category_price"], donde la llave une el canal y la categoría en minúscula (por

ejemplo, "online|running") y el valor es un árbol RBT ordenado por final_price_usd; cada nodo del árbol guarda la lista de ventas que tienen ese precio. Lo primero que se hace es armar la llave cc_key, obtener del mapa el árbol correspondiente (tree) y, con rbt.values(tree, price_min, price_max), sacar solo las listas (buckets) de los precios que están dentro del rango. Luego se recorren los buckets para agregar cada venta (sale) a result y sumar su final_price_usd en total_cost. Por último, result se ordena con al.merge_sort usando sort_crit_req_3, que ordena de menor a mayor final_price_usd y, si hay empate, de la order_date más antigua a la más reciente; si también coinciden, va primero el menor order_id.

Entrada	catalog, channel, category, price_min, price_max
Salidas	time, total, total_cost, orders
Implementado (Sí/No)	Sí. Juan Jose Rincon Santos

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	**Complejidad**
Obtener el árbol del canal y la categoría	O(1)
Obtener los buckets en el rango de precios	O(log n + k)
Recorrer los buckets, agregar a result	O(k)
Ordenar result	O(k log k)
TOTAL	***O(k log k)***

Análisis

La complejidad de este requerimiento depende de k, el número de órdenes que cumplen los tres filtros, y no del total de ventas. Como en la carga las ventas quedan agrupadas por canal y categoría, y dentro de cada grupo ordenadas por precio, no es necesario revisar ventas que no sirven: el árbol del grupo se obtiene del mapa en tiempo constante y el rango de precios se alcanza en tiempo logarítmico, de modo que todo el trabajo posterior se hace sobre las k órdenes de la respuesta. Recorrerlas y sumar su costo cuesta O(k), pero el paso que domina es el ordenamiento con merge sort, que cuesta O(k log k). Por eso la complejidad del requerimiento es O(k log k). En el peor caso, si todas las ventas fueran del mismo canal y categoría y estuvieran dentro del rango, k sería igual a n y la complejidad llegaría a O(n log n).

ANÁLISIS DEL RETO

Juan Sebastian Rojas Marin, js.rojasm123@uniandes.edu.co, 202610140

Katy Teran Mercado, k.teranm@uniandes.edu.co, 202614247

Juan Jose Rincon Santos, jj.rincons1@uniandes.edu.co, 202615789

Requerimiento <<4>>

Descripción

```python
def req_4(catalog, date_start, date_end, income_level, n):
    """
    Retorna el resultado del requerimiento 4
    """
    start = get_time()
    income_key = income_level.strip().lower()


    brands = mlp.new_map(20, 0.5)
    if date_start <= date_end:
        buckets = rbt.values(catalog["dates"], date_start, date_end)

        for i in range(al.size(buckets)):
            bucket = al.get_element(buckets, i)

            for j in range(al.size(bucket)):
                sale = al.get_element(bucket, j)

                if sale["customer_income_level"].strip().lower() == income_key:
                    brand_name = sale["brand"]
                    stats = mlp.get(brands, brand_name)

                    if stats is None:
                        stats = {
                            "brand": brand_name,
                            "count": 0,
                            "price_sum": 0.0,
                            "revenue_sum": 0.0,
                            "max_revenue_sale": None
                        }

                        mlp.put(brands, brand_name, stats)

                    stats["count"] += 1
                    stats["price_sum"] += sale["final_price_usd"]
                    stats["revenue_sum"] += sale["revenue_usd"]

                    current = stats["max_revenue_sale"]
```

```
                if (current is None or sale["revenue_usd"] > current["revenue_usd"]
                    or (sale["revenue_usd"] == current["revenue_usd"]
                        and sale["order_date"] > current["order_date"])):

                    stats["max_revenue_sale"] = sale

heap = pq.new_heap(cmp_function=sort_crit_req_4)

brand_names = mlp.key_set(brands)

for i in range(al.size(brand_names)):
    name = al.get_element(brand_names, i)
    stats = mlp.get(brands, name)

    stats["price_avg"] = (stats["price_sum"] / stats["count"])
    stats["revenue_avg"] = (stats["revenue_sum"] / stats["count"])
    priority = (stats["count"], name)
    pq.insert(heap, priority, stats)

result = al.new_list()
while not pq.is_empty(heap) and al.size(result) < n:
    al.add_last(result, pq.remove(heap))

end = get_time()

return {
    "time": delta_time(start, end),
    "total_brands": al.size(result),
    "brands": result,
}
```

```
def sort_crit_req_4(parent, child):
    """
    Mayor número de órdenes primero.
    En empate, marca alfabéticamente ascendente.
    """
    p1 = parent["priority"]
    p2 = child["priority"]

    if p1[0] != p2[0]:
        return p1[0] > p2[0]

    return p1[1] <= p2[1]
```

Este requerimiento identifica las n marcas con mayor número de órdenes de venta entre date_start y date_end para un nivel de ingresos del cliente (income_level). Se utiliza el árbol RBT catalog["dates"], ordenado por order_date, donde cada nodo guarda la lista de ventas de ese día. Lo primero que se hace es obtener con rbt.values(catalog["dates"], date_start, date_end) los buckets del rango de fechas y recorrerlos, quedándose solo con las ventas cuyo customer_income_level coincide con income_key. Para cada marca se guarda en el mapa brands un diccionario stats con count, price_sum, revenue_sum y max_revenue_sale (la orden con mayor revenue_usd; si hay empate, la más reciente según order_date). Después se calculan price_avg y revenue_avg de cada marca y se insertan en heap, una cola de prioridad

que usa sort_crit_req_4: primero la marca con más órdenes (count) y, si hay empate, el nombre de la marca en orden alfabético. Por último, se sacan de heap las n primeras marcas y se guardan en result.

Entrada	catalog, date_start, date_end, income_level, n
Salidas	time, total_brands, brands
Implementado (Sí/No)	Sí. Implementado por el grupo

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	**Complejidad**
Obtener los buckets del rango de fechas	O(log D + m)
Filtrar por income_key y actualizar stats de cada marca	O(m)
Calcular price_avg y revenue_avg e insertar en heap	O(b log b)
Calcular price_avg y revenue_avg e insertar en heap	O(n log b)
TOTAL	***O(m)***

Análisis

Este requerimiento sigue la misma idea que el 5, pero filtrando por nivel de ingresos en vez de por país, así que su costo también depende de m, el número de ventas que caen dentro del rango de fechas. Con el árbol de fechas llegamos al rango en O(log D), siendo D el número de fechas distintas, y todo lo que está por fuera de él queda descartado sin revisarlo. Dentro del rango, en cambio, no hay forma de saltarse ventas: el árbol no las separa por customer_income_level, así que tenemos que mirar cada una para saber si es del nivel pedido. Cuando una venta pasa el filtro, solo actualizamos el conteo, las sumas y la orden de mayor ingreso de su marca, y como el mapa responde en tiempo constante en promedio, el recorrido completo cuesta O(m). La cola de prioridad trabaja sobre b, el número de marcas que cumplen el filtro, que nunca pasa de 6 sin importar el tamaño del archivo, por lo que insertar y sacar marcas no cambia el orden. Por eso concluimos que la complejidad es O(m), y en el peor caso, cuando el rango cubre todas las fechas, es lineal en el total de ventas.

ANÁLISIS DEL RETO

Juan Sebastian Rojas Marin, js.rojasm123@uniandes.edu.co, 202610140

Katy Teran Mercado, k.teranm@uniandes.edu.co, 202614247

Juan Jose Rincon Santos, jj.rincons1@uniandes.edu.co, 202615789

Requerimiento <<5>>

Descripción

```python
def req_5(catalog, country, date_start, date_end, n):
    """
    Retorna el resultado del requerimiento 5: las N marcas con mejor
    calificación promedio en un país y un rango de fechas.
    """
    start = get_time()
    country_key = country.strip().lower()

    # 1. Filtrar las órdenes por fecha y país
    brands = mlp.new_map(20, 0.5)
    if date_start <= date_end:
        buckets = rbt.values(catalog["dates"], date_start, date_end)

        for i in range(al.size(buckets)):
            bucket = al.get_element(buckets, i)

            for j in range(al.size(bucket)):
                sale = al.get_element(bucket, j)

                if sale["country"].strip().lower() == country_key:
                    brand_name = sale["brand"]
                    stats = mlp.get(brands, brand_name)

                    if stats is None:
                        stats = {
                            "brand": brand_name,
                            "count": 0,
                            "rating_sum": 0.0,
                            "revenue_sum": 0.0,
                            "max_units_sale": None
                        }

                        mlp.put(brands, brand_name, stats)

                    stats["count"] += 1
                    stats["rating_sum"] += sale["customer_rating"]
                    stats["revenue_sum"] += sale["revenue_usd"]

                    # Mayor número de unidades; empate -> mayor order_id
                    current = stats["max_units_sale"]

                    if (current is None or sale["units_sold"] > current["units_sold"]
                        or (sale["units_sold"] == current["units_sold"]
                            and sale["order_id"] > current["order_id"])):

                        stats["max_units_sale"] = sale
```

```
    # 2. Promedios por marca y cola de prioridad
    heap = pq.new_heap(cmp_function=sort_crit_req_5)

    brand_names = mlp.key_set(brands)

    for i in range(al.size(brand_names)):
        name = al.get_element(brand_names, i)
        stats = mlp.get(brands, name)

        stats["rating_avg"] = (stats["rating_sum"] / stats["count"])
        stats["revenue_avg"] = (stats["revenue_sum"] / stats["count"])
        priority = (round(stats["rating_avg"], 6), name)
        pq.insert(heap, priority, stats)

    # 3. Extraer las N mejor calificadas
    result = al.new_list()
    while not pq.is_empty(heap) and al.size(result) < n:
        al.add_last(result, pq.remove(heap))

    end = get_time()

    return {
        "time": delta_time(start, end),
        "total_brands": al.size(result),
        "brands": result,
    }
```

```
def sort_crit_req_5(parent, child):
    """
    Mayor calificación promedio primero.
    En empate, marca alfabéticamente ascendente.
    """
    p1 = parent["priority"]
    p2 = child["priority"]

    if p1[0] != p2[0]:
        return p1[0] >= p2[0]

    return p1[1] <= p2[1]
```

Este requerimiento identifica las n marcas con mejor calificación promedio en un país (country) entre date_start y date_end. En la carga de datos se crea el árbol RBT catalog["dates"], ordenado por order_date, donde cada nodo guarda la lista de ventas de ese día. Lo primero que se hace es obtener con rbt.values(catalog["dates"], date_start, date_end) los buckets del rango de fechas y recorrerlos, quedándose solo con las ventas cuyo country coincide con country_key. Para cada marca se guarda en el mapa brands un diccionario stats con count, rating_sum, revenue_sum y max_units_sale (la orden con más units_sold; si hay empate, la de mayor order_id). Después se calculan rating_avg y revenue_avg de cada marca y se insertan en heap, una cola de prioridad que usa sort_crit_req_5: primero la mayor

rating_avg (redondeada a 6 decimales) y, si hay empate, el nombre de la marca en orden alfabético. Por último, se sacan de heap las n primeras marcas y se guardan en result.

Entrada	catalog, country, date_start, date_end, n
Salidas	time, total_brands, brands
Implementado (Sí/No)	Sí. Implementado por el grupo

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	**Complejidad**
Obtener los buckets del rango de fechas	O(log D + m)
Filtrar por country_key y actualizar stats de cada marca	O(m)
Calcular rating_avg y revenue_avg e insertar en heap	O(b log b)
Sacar las n mejores marcas de heap	O(n log b)
TOTAL	***O(m)***

Análisis

El tiempo de este requerimiento depende de m, el número de ventas que caen dentro del rango de fechas. Gracias al árbol de fechas que armamos en la carga, llegamos al rango en O(log D), siendo D el número de fechas distintas, y no tenemos que revisar las ventas que están por fuera de él. Las que quedan dentro sí las recorremos todas, porque el árbol no las separa por país y la única forma de saber cuáles son de country es mirarlas una por una. Cada una de esas ventas solo actualiza los datos de su marca en el mapa, lo que toma tiempo constante en promedio, así que este recorrido cuesta O(m) y es lo que más pesa. El trabajo con el heap depende de b, el número de marcas que cumplen el filtro, que en los datos nunca pasa de 6 y no crece con el archivo, así que su costo es prácticamente constante. Por eso concluimos que la complejidad del requerimiento es O(m). En el peor caso, si el rango abarca todas las fechas, m es igual al total de ventas y el requerimiento es lineal.

Para comprobarlo, corrimos el requerimiento varias veces con Germany, del 2021-01-01 al 2023-12-31 y n = 5, en los tres archivos. En promedio obtuvimos 5.46 ms con el small, 7.31 ms con el medium y 12.35 ms con el large. De small a large, m pasa de unas 2 500 a unas 10 000 ventas, casi cuatro veces más, pero el tiempo solo se multiplica por 2.3. Esto no va en contra del O(m). Al ser tiempos de pocos milisegundos, una buena parte se va en trabajo que no depende de m, como crear el mapa de marcas y el heap, y además la medición varía un poco entre ejecuciones. Esa parte fija pierde peso a medida que el archivo crece, y por eso del medium al large el tiempo ya crece de forma casi proporcional a m, tal como esperábamos.

ANÁLISIS DEL RETO

Juan Sebastian Rojas Marin, js.rojasm123@uniandes.edu.co, 202610140

Katy Teran Mercado,k.teranm@uniandes.edu.co, 202614247

Juan Jose Rincon Santos, jj.rincons1@uniandes.edu.co, 202615789

Requerimiento <<6>>

Descripción

```python
def req_6(catalog, date_start, date_end, price_min, price_max, n):
    """
    Retorna el resultado del requerimiento 6: las N marcas más consistentes
    (menor desviación estándar del discount_percent) para un rango de fechas
    y un rango de precios base.
    """
    start = get_time()

    # 1. Filtrar por fecha (RBT) y precio base; agrupar por marca
    brands = mlp.new_map(20, 0.5)
    if date_start <= date_end and price_min <= price_max:
        buckets = rbt.values(catalog["dates"], date_start, date_end)
        for i in range(al.size(buckets)):
            bucket = al.get_element(buckets, i)
            for j in range(al.size(bucket)):
                sale = al.get_element(bucket, j)
                if price_min <= sale["base_price_usd"] <= price_max:
                    brand_sales = mlp.get(brands, sale["brand"])
                    if brand_sales is None:
                        brand_sales = al.new_list()
                        mlp.put(brands, sale["brand"], brand_sales)
                    al.add_last(brand_sales, sale)

    # 2. Estadísticas por marca y cola de prioridad (min-heap)
    heap = pq.new_heap(True)
    brand_names = mlp.key_set(brands)
    total_brands = al.size(brand_names)
    for i in range(total_brands):
        name = al.get_element(brand_names, i)
        stats = brand_discount_stats(name, mlp.get(brands, name))
        # Menor desviación primero; empate -> menor promedio; luego nombre
        priority = (round(stats["std"], 9), round(stats["mean"], 9), name)
        pq.insert(heap, priority, stats)
```

```
    # 3. Extraer las N más consistentes
    result = al.new_list()
    while not pq.is_empty(heap) and al.size(result) < n:
        al.add_last(result, pq.remove(heap))

    end = get_time()
    return {
        "time": delta_time(start, end),
        "total_brands": total_brands,
        "brands": result,
    }
```

```
def brand_discount_stats(name, sales):
    """
    Calcula número de órdenes, promedio y desviación estándar poblacional
    del discount_percent, y la orden con descuento más cercano al promedio
    (empate -> menor order_id).
    """
    count = al.size(sales)
    total = 0.0
    for i in range(count):
        total += al.get_element(sales, i)["discount_percent"]
    mean = total / count

    squares = 0.0
    closest = None
    best_diff = None
    for i in range(count):
        sale = al.get_element(sales, i)
        x = sale["discount_percent"]
        squares += (x - mean) ** 2
        diff = abs(x - mean)
        if (closest is None
                or diff < best_diff - 1e-9
                or (abs(diff - best_diff) <= 1e-9
                    and order_number(sale["order_id"]) < order_number(closest["order_id"]))):
            closest = sale
            best_diff = diff
    std = math.sqrt(squares / count)

    return {
        "brand": name,
        "count": count,
        "mean": mean,
        "std": std,
        "closest": closest,
    }
```

Identifica las N marcas más consistentes, es decir, con menor desviación estándar del discount_percent, dentro de un rango de fechas y un rango de precio base. Se usa el RBT dates, construido en la carga, con llave la fecha y valor un array_list con las ventas de ese día. Con rbt.values se obtienen solo las ventas dentro de [date_start, date_end]. Al recorrerlas se filtra por base_price_usd y cada venta se agrupa por marca en una tabla de hash auxiliar Linear Probing: la llave es la marca y el valor es la lista de sus ventas. Luego, por cada marca, se calculan el promedio y la desviación estándar del descuento, y se insertan en un min-heap con prioridad (desviación, promedio, nombre), lo que resuelve los empates. Al final se extraen las primeras N marcas, o todas si N supera el total.

Entrada	catalog, date_start, date_end, price_min, price_max, n
Salidas	time, total_brands, brands (con el promedio y la desviación estándar del descuento de cada marca)
Implementado (Sí/No)	Implementado por el grupo 8

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	**Complejidad**
Consultar el rango de fechas en el RBT dates (rbt.values)	O(log D + r)
Recorrer las s ventas del rango y filtrar por precio base	O(s)
Agrupar cada venta que cumple en la tabla de marcas (mlp.get / mlp.put)	O(1) promedio por venta, entonces O(s)
Calcular promedio y desviación estándar de las b marcas	O(s) en total
Insertar las b marcas en el min-heap	O(b log b)
Extraer las primeras N marcas (N ≤ b)	O(N log b)
TOTAL	***O(log D + s + b log b)***

D = fechas distintas, r = fechas dentro del rango, s = ventas dentro del rango de fechas, b = marcas distintas que cumplen ambos filtros.

Análisis

Análisis de resultados de la implementación, tener cuenta las pruebas realizadas y el analisis de complejidad.

El costo está dominado por el recorrido de las s ventas dentro del rango de fechas. El RBT permite llegar a ese intervalo en O(log D + r) en lugar de revisar todo el archivo, y el filtro de precio se aplica en esa misma pasada. La tabla Linear Probing agrupa por marca en O(1) promedio, y con un factor de carga de 0.5 el sondeo casi siempre se resuelve en las primeras posiciones. Se usó un heap en vez de ordenar toda la lista porque solo se necesitan las N más consistentes: cada extracción cuesta O(log b), y con N pequeño eso sale más barato que ordenar las b marcas. La tupla de prioridad resuelve los empates sin un criterio de comparación aparte